

Implementación del patrón Command en el sistema de Citas Médicas

1.Introducción

El patrón de diseño Command nos permite ejecutar operaciones sin conocer los detalles de la implementación de la misma. Las operaciones son conocidas como comandos y cada operación es implementada como una clase independiente que realiza una acción muy concreta, para lo cual, puede o no recibir parámetros para realizar su tarea. Una de las ventajas que ofrece este patrón es la de poder crear cuántos comandos requerimos y encapsularlos bajo una interfaz de ejecución.

2.¿Qué es el patrón Command?

Command es un patrón de diseño de comportamiento que convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.

3.Participantes

- ICommand: Interfaz que describe la estructura de los comandos, la cual define el método de ejecución genérico para todos los comandos.
- Concrete Command: Representan los comandos concretos, éstos deberán heredar de ICommand. Cada una de estas clases representa un comando que podrá ser ejecutado de forma independiente.
- Command Manager: Este componente nos servirá para administrar los comandos que tenemos disponibles en tiempo de ejecución, desde aquí podemos solicitar los comandos o registrar nuevos.
- Invoker: El invoker representa la acción que dispara alguno de los comandos.

4.Problema

Para implementar el patrón Command en tu proyecto, podemos enfocarnos en encapsular acciones como comandos que puedan ejecutarse, deshacerse o almacenarse fácilmente, un ejemplo es registrar un usuario.

Al registrar un usuario en el evento del botón tenemos instrucciones muy amplias, lo que podemos hacer en este caso es crear varias clases o subclases para cada acción donde se utilice el botón. Estas subclases contendrán el código que deberá ejecutarse con el clic en un botón.

5.Solución

El buen diseño de software a menudo se basa en el principio de separación de responsabilidades, lo que suele tener como resultado la división de la aplicación en capas. El ejemplo más habitual es tener una capa para la interfaz gráfica de usuario (GUI) y otra capa para la lógica de negocio. La capa GUI es responsable de representar una bonita imagen en pantalla, capturar entradas y mostrar resultados de lo que el usuario y la aplicación están haciendo. Sin embargo, cuando se trata de hacer algo importante, como calcular la trayectoria de la luna o componer un informe anual, la capa GUI delega el trabajo a la capa subyacente de la lógica de negocio.

6.Diagrama UML

7.Código de implementación

Interface Comando:

```
public interface Comando {  
    void ejecutar();  
}
```

Clase InvocadorRegistro:

```
public class InvocadorRegistro {  
    private Comando command;  
  
    public void setCommand(Comando command) {  
        this.command = command;  
    }  
  
    public void ejecutarRegistro() {  
        if (command != null) {  
            command.ejecutar();  
        }  
    }  
}
```

Clase RegistrarUsuarioComando:

```
public class RegistrarUsuarioComando implements Comando {  
    private Usuario usuario;  
    private IRegistroUsuario registro;  
  
    public RegistrarUsuarioComando(Usuario usuario, IRegistroUsuario registro) {  
        this.usuario = usuario;  
        this.registro = registro;  
    }  
  
    @Override  
    public void ejecutar() {  
        try {  
            boolean exito = registro.registrar(usuario);  
        } catch (SQLException e) {  
            System.err.println("Error SQL: " + e.getMessage());  
        }  
    }  
}
```